1 Introduction

This evidence looks at finding an automaton to accept a provided language. The resulting automaton can than be either modeled using Prolog or REGEX. The language chosen for this evidence posesses the alphabet $\Sigma = \{0, 1, 2\}$, needs to end in either "111" or "122" to transition into an accepting state and prohibits the occurence of "211".

2 Automaton

As [1] mention, (deterministic) finite automaton are useful models for different tasks, including lexical analysis. When interpreting finite automatons as transition graphs, this can be easily visulaized. A string can be broken down into each letter, which can be used to follow a graph into a specific node/ state. In [1] a deterministic finite automaton is defined as a quintuple $A = (Q, \Sigma, \delta, q0, F)$ where

- Q defines a finite set of states,
- Σ a finite alphabet,
- δ a transition function,
- q0 the inital state and
- F the accepting states.

An input gets accepted if, after passing the string, it ends up in an accepting state, otherwise it is rejected. The alphabet Σ is the same as mentioned in section 1 and the other elements are yet to be determined.

While [1] also mention the existence of non-deterministic finite automaton, due to the small size and lower complexity of the language, a deterministic model approach was chosen. As [1] explain, a non-deterministic model has to be converted to a deterministic model, in order to be implementable. As using a deterministic model was possible, this option was more reasonable.

Each string should start out at the same entry state E. From there one can derive the different conitions path, the automaton needs to recognize. In fig. 1 the different paths are depicted. The path s1e-s1f shows the accepted ending sequence "111", s1e-s2f models the accepting end string "122" and s3e-s3f show the rejected sequence "211". These are also all the required states, as any other sequence has no influence on acceptance or reejction and can lead back to E. With this F is defined as $F = \{s1f, s2f\}$. In fig. 2, the full model is shown with all connections. Important to note is, that each node needs to model each possible alphabet transition. Interesting observations include, that each 0 loops back to node E as 0 is no part of any important sequence, that once s3f is reached, all further transitions loop back to itself. Once this node has been reached, the string cannot be accepted. Further interesting to observer are

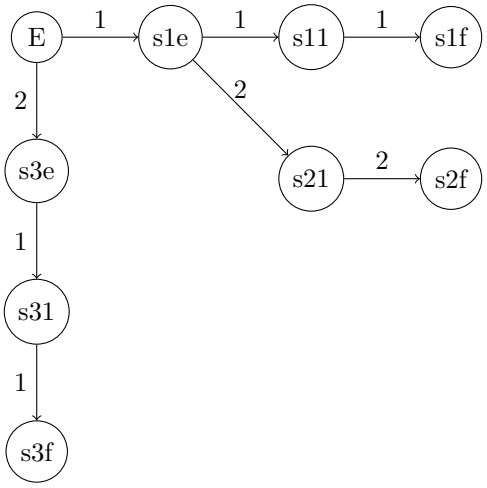

Figure 1: Base

the transitions between different branches, as each subsequence in a branch can potentially be the starting point of a different sequence.

3 Prolog

One method to implement the model is using Prolog. Due to the nature of prolog, the model can be easily implemented using the visual representation. It is only necessary to define the transitions and acceptance states:

```
move(e, e, 0).
move(e, s1e, 1).
move(e, s3e, 2).

move(s1e, e , 0).
move(s1e, s11 , 1).
move(s1e, s21 , 2).

move(s11, e , 0).
move(s11, s1f , 1).
move(s11, s21 , 2).

move(s1f, e , 0).
move(s1f, s1f , 1).
move(s1f, s21 , 2).

move(s21, e , 0).
move(s21, s31 , 1).
```

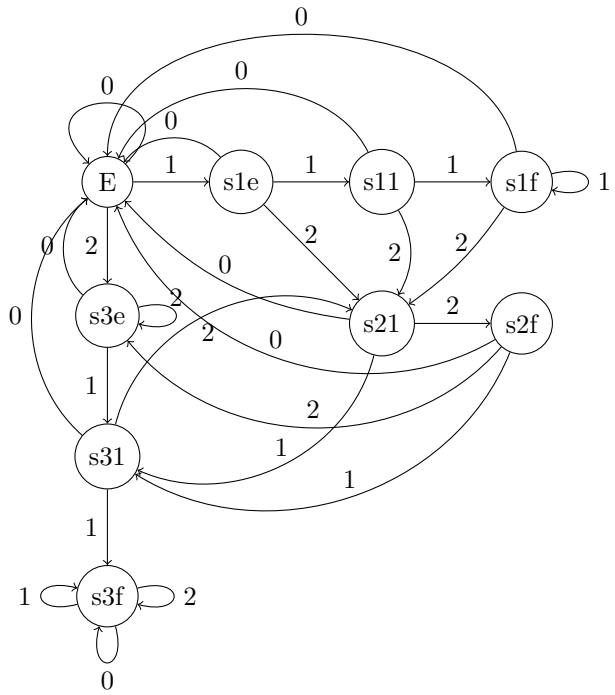

Figure 2: Base

```
move(s21, s2f, 2).

move(s2f, e, 0).
move(s2f, s31, 1).
move(s2f, s3e, 2).

move(s3e, e, 0).
move(s3e, s31, 1).
move(s3e, s2e, 2).

move(s31, e, 0).
move(s31, s3f, 1).
move(s31, s21, 2).

move(s3f, s3f, 0).
move(s3f, s3f, 1).
move(s3f, s3f, 2).

accepting_state(s1f).
accepting_state(s2f).
```

The parser provieded in the lecture is used to parse the string letter by letter.

```
parseDFA(InputList) :-
    parseDFAHelper(InputList, e).
parseDFAHelper([], CurrentState) :-
    accepting_state(CurrentState),
    write('Accepted'), nl;
    write('Rejected'), nl.
parseDFAHelper([Symbol|Rest], CurrentState) :-
    move(CurrentState, NextState, Symbol),
    parseDFAHelper(Rest, NextState).
```

The helper function separates out each symbol and uses the symbol to match a *move* clause definition to obtain the new state. The full script, including the test cases can be found in "dfa.pl".

If given a string of length n, the tail recursion algorithm looks at every symbol only once. Therefore, the algorithm is in $O(n)$.

4 REGEX

"REGEX", short for "regular expression" is according to [1] an algebraic description of languages. They further proof that any language that can be represented using a deterministic finite automaton, can also be represented using regular expressions. Designing a regular expression is different than designing an automaton. Instead of thinking in individual states, one starts out by breaking the language down into individual patterns and strings them together. In

this example we can start out with the easiest condition: The string has to and in either "111" or "122". In regex this can be denoted as

$$(111|122)\$$$

Here the "$" enforces the string to end in the prior pattern. Before this pattern, any pattern that doesn not contain "211" can be used. This case can be broken down into 6 different patterns that can be repeated infinetly to build any valid pattern using $*$. Combined with the previous pattern and the addition of a special case, the regular expression results in:

$$\hat{}(0|1|20|22|212|210)*(2122|111|122)\$.$$

Note the special ending sequence "2122". As the first part can have any number of 1s and the sequence can end in "111", adding a single "2" in the first part, would allow for the acceptance of invalid patterns. The only case where "21" would not raise a potential concern is when the string ends in "122". Thus the ending sequence "2122" has to be matched separatly. The forces the beginning of string to also start with any of those patterns, forcing REGEX to consider the entire string for matching and not mearly parts.

5 Tests

To test both implementations, test have been derived to test various cases. The cases have been divided into two catergories: String that should be accepted and string that should be rejected.

5.1 Accepted

Here the cases 1. and 2. validate the base cases, where only the ending sequence occurs. Cases 3., 4. and 6. test more complex cases, with longer strings. Cases 5. and 7. are specifically designed to test switching inbetween sequences, for example in 5., although the ending sequence "111" fullym appears, the actual ending sequence "122" has to be identified.

1. 111
2. 122
3. 1122
4. 00120122
5. 111111122
6. 01201201111
7. 112122

5.2 Rejected

The rejection cases test either the end sequence such as cases 1. and 4. or the occurence of "211" which is tested in all other cases. Case 3. shows an interesting problem for REGEX, as the sequence "2122" needs to be accepted, but the sequence "2111" needs to be rejected.

1. 1112
2. 211
3. 01201202111
4. 00121222
5. 001211222
6. 11112110111
7. 2110102001221200122

While the tests don't enumerte every single possible case, they test every switch of sequence/ branch and some combinations of those. These tests and parts of tests can be used to construct any other string.

6 Comparison and Learnings

References

[1] John E Hopcroft, Rajeev Motwani, and Jeffrey D Ullman. *Introduction to automata theory, languages, and computation, 3rd edition*. Pearson Education, Inc, 2007.